

Ostbayerische Technische Hochschule Amberg/Weiden
Department of Electrical Engineering, Media, and
Computer Science

Prof. Dr. Ulrich Schäfer

Project Mobile & Ubiquitous Computing Summer
Semester 2026

Group 03: Simon Völkl & Johannes Schieder

Courses of study: Artificial Intelligence, Industry-4.0-Informatics

June 29, 2026



Figure 1: Project Logo of the Bite-Bound Game. Designed by Simon Völkl.



Figure 2: Waveshare ESP32-S3 1.69inch Touch Display. Source: waveshare.com

Contents

1	Introduction	3
1.1	Introduction	3
1.2	Mission Statement	3
1.3	Motivation	3
1.4	Document Structure	3
2	Technical Concept	3
2.1	High-Level Architecture	3
2.2	Components	4
3	Hardware	4
3.1	ESP32-S3 Microcontroller	4
3.2	Concurrency	4
3.3	Sensor Data Acquisition	6
3.4	Physics Simulation & Game Logic	6
3.5	Display	7
3.6	MQTT Communication	8
4	Node-RED	8
4.1	Workflow	8
4.2	Dashboard & User Input	8
5	Android App	8
5.1	Software Components, Classes, Views	8
5.2	Dashboard & User Input	8
6	Project Management	8
6.1	Project Planning	9
6.2	Task Dependencies	10
6.3	Task Distribution	10
7	Evaluation	11
7.1	Problems & Solutions	11
7.2	Code Coverage	12
8	Summary and Future Work	12
A	Prompts	12
B	ESP32 Code	12
B.1	esp32.ino	12
B.2	Headers	12
B.3	Source Files	12

Abstract

1 Introduction

1.1 Introduction

1.2 Mission Statement

1.3 Motivation

1.4 Document Structure

2 Technical Concept

The technical concept of BiteBound is designed to ensure real-time responsiveness, modularity, and thread safety. By leveraging the dual-core capabilities of the ESP32-S3, the system separates latency-sensitive gameplay and rendering tasks from time-intensive network operations.

2.1 High-Level Architecture

The architecture relies on an asynchronous, event-driven design. The ESP32-S3 runs two primary execution threads across its dual cores to balance the computational workload. Core 1 is designated as the Game Core, measuring the sensor data, executing the physics engine, handling the game logic, and updating the display at a constant 50 Hz. Core 0 acts as the Network Core, managing secure TLS connections, MQTT messaging, and transmitting telemetry at circa 2 Hz.

Communication between the embedded hardware and the external control interfaces is bridged through the centralized MQTT broker HiveMQ. External clients communicate bidirectionally with the device. They send game-related commands to a dedicated topic, while subscribing to a telemetry topic to monitor real-time variables.

The system architecture is depicted in Figure 3, illustrating the interactions between the ESP32-S3 and the frontend dashboards (see `diagrams/out/`).

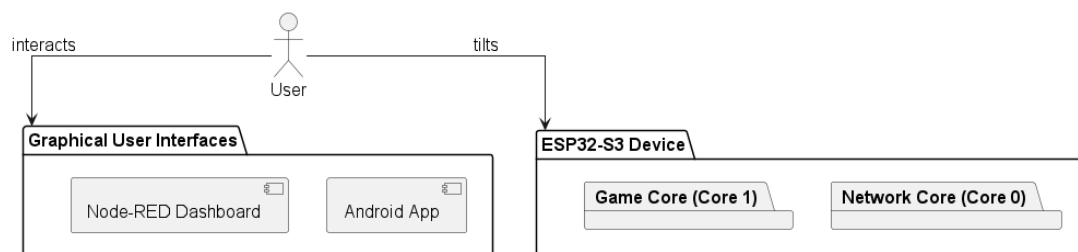


Figure 3: System architecture diagram displaying the user interactions with the ESP32-S3 and the frontend dashboards.

2.2 Components

The system architecture is divided into three primary components:

- **Hardware (ESP32-S3):** The core of the system is the Waveshare ESP32-S3 development board equipped with a 1.69-inch touch LCD, a 6-axis inertial measurement unit (IMU), and a buzzer. It hosts the procedural sensor manager, maze generator, 2D physics simulation, game engine, display logic, and network communication stack.
- **Android Application:** A native mobile interface written in Kotlin. It receives telemetry data and sends control commands to the ESP32-S3 via MQTT.
- **Node-RED Dashboard:** A flow-based UI-Builder dashboard that mirrors the application logic of the Android app.

3 Hardware

The firmware of the BiteBound platform is written in C++ using the Arduino framework [?]. The code operates on a dual-core microcontroller to separate timing-sensitive game updates from network Input/Output (I/O) activities.

3.1 ESP32-S3 Microcontroller

The core processing unit is a Waveshare ESP32-S3 1.69-inch Touch LCD development board [?]. Onboard peripherals integrated on the board include a QMI8658 six-axis Inertial Measurement Unit (IMU), a Real-Time Clock (RTC), and an ST7789-driven 240×280 pixel Liquid Crystal Display (LCD). Dedicated hardware pins and interface buses (I2C for sensors, SPI for the display) are managed centrally via the `config.h` configuration file [?].

3.2 Concurrency

Execution is distributed across both processor cores using FreeRTOS tasks to maintain stable game loop execution rates [?]:

- **Core 1 (Game Core):** Runs the main application thread executing the game loop at approximately 50 Hz (20 ms loop budget). This core is responsible for sensor readings, physics steps, collision resolution, game engine updates, and display rendering. It is designed to be non-blocking and deterministic to ensure smooth gameplay.
- **Core 0 (Network Core):** Executes the background network task (`vNetworkTask`) at approximately 2 Hz. This core handles secure TLS handshakes, manages the MQTT client connection, and serializes and publishes telemetry data.

Inter-process communication (IPC) between cores utilizes two distinct mechanisms to ensure thread safety and avoid race conditions [?]:

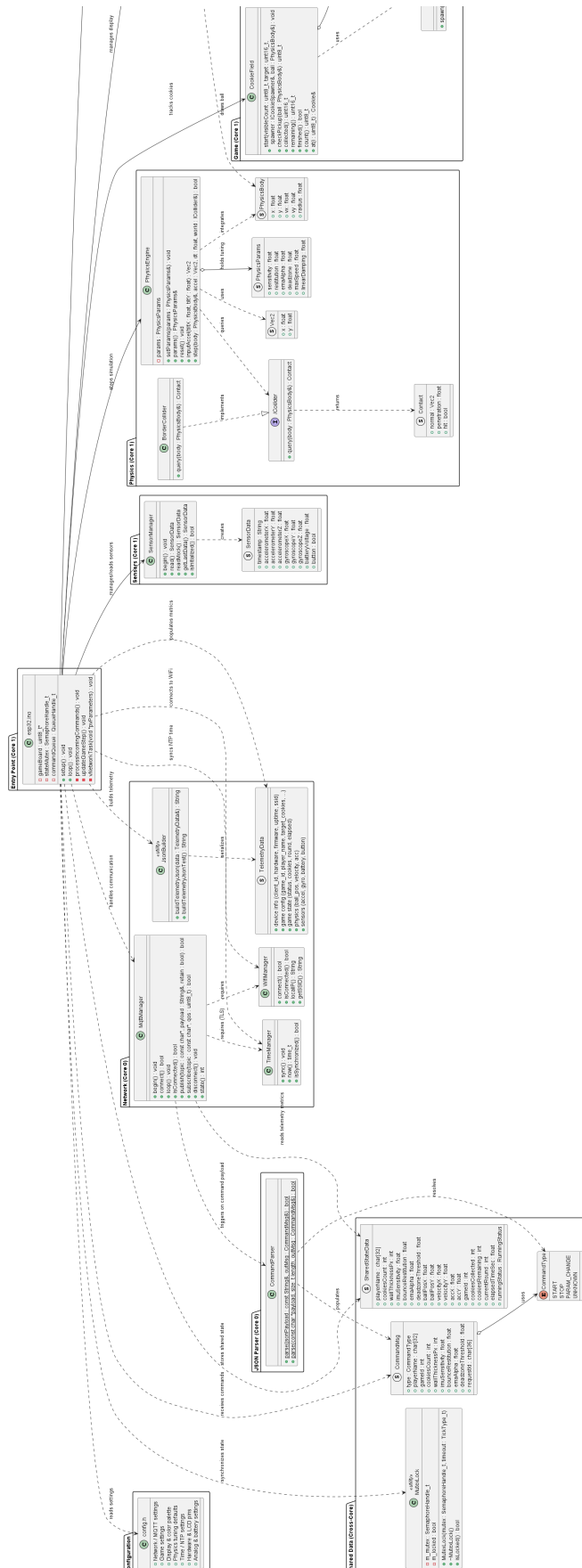


Figure 4: Hardware architecture UML diagram.

- **Mutex Synchronization:** The global state structure (`SharedStateData`) is protected with a FreeRTOS semaphore. Access is managed through a C++ Resource Acquisition Is Initialization (RAII) helper class (`MutexLock`) to automate the lock release cycle and avoid deadlock states [?].
- **Command Queue:** Unidirectional commands from Core 0 are transferred to Core 1 using a thread-safe FreeRTOS queue (`commandQueue`) with a length of 10. This structure prevents network tasks from causing latency or stalling the rendering pipeline [?].

3.3 Sensor Data Acquisition

Onboard data collection is managed by the `SensorManager` class, which reads physical values from the QMI8658 IMU over I2C and samples battery voltage through an internal Analog-to-Digital Converter (ADC). The raw accelerometer (a_x, a_y, a_z) and gyroscope ($\omega_x, \omega_y, \omega_z$) readings are structured inside the `SensorData` format.

To filter out high-frequency noise and mechanical vibrations, the raw tilt inputs are smoothed using an Exponential Moving Average (EMA) filter [?]:

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1} \quad (1)$$

where S_t is the filtered value, Y_t is the current sensor input, and α is the smoothing coefficient (`ema_alpha` in configuration). A deadzone threshold is then applied to the output; inputs falling below this range are registered as zero, preventing drifting movement when the controller is resting.

3.4 Physics Simulation & Game Logic

The two-dimensional physics simulation is implemented inside the `PhysicsEngine` class. It is parameterized via the `PhysicsParams` structure, enabling real-time adjustments. The ball is modeled as a `PhysicsBody` with position $\vec{x}$, velocity $\vec{v}$, and radius r .

The engine uses semi-implicit Euler integration [?] with a constant time step ($dt = 0.02$ s) to update kinematics:

$$\vec{v}_{t+1} = \vec{v}_t + \vec{a} \cdot dt \quad (2)$$

$$\vec{x}_{t+1} = \vec{x}_t + \vec{v}_{t+1} \cdot dt \quad (3)$$

where $\vec{a}$ is the scaled acceleration derived from the filtered tilt values. The velocity is capped at a configurable `maxSpeed` threshold. To prevent tunneling errors when dealing with fast movements or thin boundaries, sub-stepping is utilized during collision detection.

Collisions with the environment are resolved via the `ICollider` strategy interface. The `BorderCollider` class checks for rectangular screen boundary crossings, while the `MazeCollider` evaluates pixel-level collisions against the generated layout grid. Upon detecting a collision, the engine extracts the normal vector $\hat{n}$ and calculates the updated velocity according to a restitution model [?]:

$$\vec{v}_{\text{new}} = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n} \quad (4)$$

where $e \in [0, 1]$ represents the bounce restitution factor. The tangential velocity remains unaffected, permitting the ball to slide along walls.

Collectibles are modeled using the `CookieField` class. When the physical body of the ball overlaps with an active cookie coordinate within a configured pickup radius, the score is incremented and the `ICookieSpawner` strategy interface instantiates a new cookie. The `RectCookieSpawner` places cookies within random bounds for the open field, whereas the `MazeCookieSpawner` identifies walkable coordinates within the maze layout using generated cell lists.

3.5 Display

The rendering of the interface is performed by the `GraphicsManager` using the Adafruit GFX-compatible `Arduino_GFX` library [?].

Transmitting a full 16-bit color frame (240×280 pixels) requires writing 131.25 KB of data. Since SPI transfers one bit per clock cycle, a 27 MHz bus provides a theoretical throughput of 27 Mbit/s, resulting in a frame transmission time of approximately 40 ms, derived from the ratio between total frame size and bus bandwidth. This exceeds the 20 ms limit of the 50 Hz game loop. To achieve a consistent frame rate, the display module utilizes a hybrid update scheme:

Updating a 240×280 display in 16-bit color (RGB565) requires pushing **131.25 KB** of data per frame. Over a standard, stable 27 MHz SPI bus, a full frame transmission takes **40ms**, which physically violates the 20ms constraint of a 50Hz game loop.

To circumvent this hardware bottleneck, the `GraphicsManager` implements a hybrid rendering architecture utilizing the dirty rectangles technique [?]:

- **Full Redraw:** Executed at the start of each round or when the game state changes. The manager clears the entire screen, draws the HUD, and renders the static maze layout. To optimize SPI bandwidth, horizontal spans of identical pixel states are drawn using a Run-Length Encoding (RLE) [?] approach instead of individual pixels.
- **Partial Redraw:** Used during active game updates. On every tick, the manager clears only the bounding box representing the ball's previous position. It recovers the background texture or any overlapping cookie graphics, then draws the ball at its new coordinate. This limits the active data transfer to approximately 1 KB per frame, keeping SPI communication time under 1 ms.

Volatile HUD elements, such as the score and elapsed time, are cached and updated only when their internal values change.

3.6 MQTT Communication

The network layer connects to a HiveMQ Cloud broker over an encrypted TLS connection [?]. Network credentials and SSL certificates are compiled from the git-ignored `wifi_mqtt_secrets.h` file. Certificate validation is supported by synchronization with the NTP server pool via the TimeManager during initialization.

The device establishes two main communication paths:

- **Command Topic** (`mauc2026/group_03/game/command`): Receives control payloads from external dashboards, as illustrated in Figure 5. The incoming JSON content is processed by the static `CommandParser` class to adjust gameplay states or update physical and visual parameters on the fly [?].
- **Telemetry Topic** (`mauc2026/group_03/game/telemetry`): Published telemetry data to dashboards at a 2 Hz rate, as shown in Figure 6. The `JsonBuilder` class structures the combined hardware information, game progress, configuration records, physics parameters, and raw sensor readings into a unified JSON format.

All communication packets default to Quality of Service 1 (QoS 1) with the retain flag set to false to avoid processing stale telemetry.

4 Node-RED

[?]

4.1 Workflow

4.2 Dashboard & User Input

5 Android App

5.1 Software Components, Classes, Views

5.2 Dashboard & User Input

6 Project Management

The BiteBound project was executed by a two-person team, with each member contributing to all aspects of the project. The lead developers for each component are indicated in Table 1, while a reviewer was assigned to each task to ensure code quality and maintainability, as stated in Section 6.3.

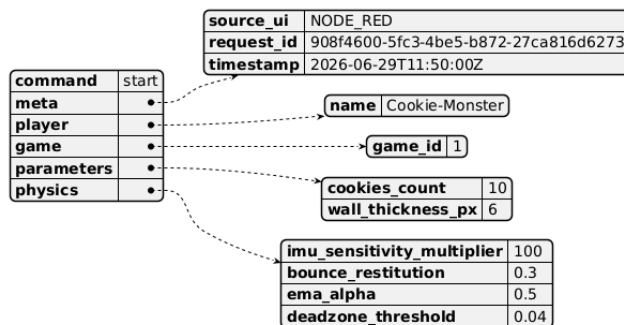


Figure 5: Dashboard JSON Command Payload to the ESP32-S3.

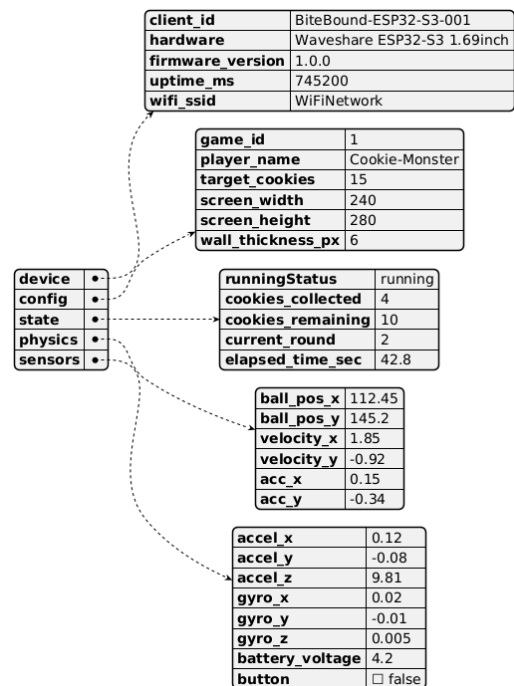


Figure 6: ESP32-S3 JSON telemetry payload to the dashboards.

6.1 Project Planning

The development of BiteBound followed an incremental path, prioritizing the definition of interface contracts and network infrastructure before implementing complex local logic such as the physics simulation and graphics rendering. The project was structured into four distinct milestones to manage dependencies and verify integration at each phase.

- Milestone 1: Architecture and Basic Setup** This phase focused on creating the repository structure, planning extensive software architecture, establishing development environments, and configuring the basic ESP32-S3 firmware skeleton. Essential network modules (WiFi connectivity, secure MQTT communication via HiveMQ Cloud, NTP-based time synchronization) were implemented. Defining the standard JSON schemas for telemetry and command topics early in the timeline allowed the basic setups of the Android application and Node-RED dashboards to proceed in parallel with the firmware development.
- Milestone 2: Hardware Implementation** During this milestone, the core offline mechanics of the ESP32 firmware were established. The 2D physics engine and cookie collection logic were designed first, followed by the integration of the Sensor Manager and maze generation. The graphics subsystem was then built, including an evaluation of display libraries. Finally, the firmware was fully orchestrated by the combination of the Game Engine for the game modes and

the FreeRTOS-based dual-core task management using JSON Parser and .

- **Milestone 3: UI Enhancement and Refinement** This phase concentrated on system integration, user interface polishing, and debugging. The Android application and Node-RED dashboards were updated to support the finalized game orchestration logic, enabling dynamic command and parameter handling. The project's visual identity with the logo, color scheme and assets was finalized, while last tests and bug-fixes were conducted to address firmware issues.
- **Milestone 4: Documentation** The final phase was dedicated to finalizing the technical report as documentation, enhancing the usage guide, and preparing the project for submission.

6.2 Task Dependencies

The decoupled software architecture introduced several critical dependencies: IMU calibration fed into physics input, JSON schema definitions enabled dashboard integration, and dual-core allocation required thread-safe state protection before merging the network and game codebases. These dependencies ensured that network managers and telemetry contracts were established first, physics and sensor processing were validated before graphics integration, and the multi-core architecture was introduced only after both stacks were independently stable.

6.3 Task Distribution

To coordinate development and ensure progress, tasks were tracked and managed using a GitHub Project board integrated directly with the code repository. This allowed issues to be linked to commits and Pull Requests (PRs).

To maintain code quality and structural integrity across the shared codebase, the development followed several guidelines:

- **Branch Protection:** Direct commits to the main branch were restricted. All feature updates and bug fixes were developed on separate branches.
- **Code Review:** Merging any branch into main required a Pull Request accompanied by the other team member's review and approval.
- **CI Pipeline:** A GitHub Actions pipeline was configured to automatically verify the compilation of the ESP32 firmware for both test and production builds (see `.github/workflows/ci.yaml`)
- **Code Coverage Targets:** A unit test coverage of about 60% was targeted for the core ESP32 modules, while the Android application aimed for basic unit tests. Node-RED flows were excluded from automated testing due to their visual nature (see `esp32/test-coverage.txt`).
- **Docstrings:** All public methods and classes were documented with docstrings to enhance code readability and accessibility.

Transparency note: The unit test writing and docstring generation were partly assisted by AI, as documented in Section A. The division of responsibilities between the two team members is outlined in Table 1.

Table 1: Task Distribution Matrix Grouped

Milestone	Task	Description	S. Völkl	J. Schieder
Milestone 1: Architecture & Setup	Architecture	Software Architecture, Protocol Definitions, Repository Setup	Lead	Lead
	Networking	Implementation of WiFi, MQTT and Time synchronization.	Lead	Review
	Android App	Basic Android app, MQTT transport via HiveMQ.	Review	Lead
	Node-RED	Basic Node-RED setup, UI-Builder Dashboard.	Lead	Review
Milestone 2: Hardware Implementation	Tests	Automated AUnit Tests for hardware components.	Lead	Lead
	Physics	Core 2D physics engine, collisions, and cookies.	Review	Lead
	Sensor Manager	Configuration of QMI8658 IMU, battery and button inputs.	Lead	Review
	Maze	Iterative random DFS.	Lead	Review
	Graphics	Library comparison, Driver setup, ST7789 config, rendering.	Lead	Review
	JSON Builder and Parser	JSON serialization of game state and deserialization of commands.	Lead	Review
	Concurrency	FreeRTOS multi-core tasking, queues, and mutex sharing.	Lead	Review
	Game Engine Game Orchestration	Game modes, engine and start. Orchestrated concurrency and game engine in Arduino.	Review Lead	Lead Review
Milestone 3: UI & Refinement	Branding	Designing the logo, color scheme, and visual assets.	Lead	Lead
	Enhanced Android App	Updating Android App to support new features and improvements.	Lead	Lead
	Enhanced Node-RED UI	Updating Node-RED UI to support new features and improvements.	Lead	Review
	Manual Testing	Manual code testing and bug fixes.	Lead	Lead
Milestone 4: Documentation	Technical Report	Finalizing the LaTeX report, code docs, and usage guides.	Lead	Review
	Presentation	Preparing and delivering the final presentation.	Lead	Lead

7 Evaluation

7.1 Problems & Solutions

[?] should not be used anymore. So we decided on using [?] instead for our dashboard.

7.2 Code Coverage

8 Summary and Future Work

A Prompts

For transparency and reproducibility, the used AI tool prompts are attached in this section. In general, the model Google Gemini Flash 3.5 Thinking was used for all prompts.

B ESP32 Code

The main control logic for the ESP32 microcontroller, including the core game loop, hardware abstraction, and network communication, is presented in this section.

As the requirements state that the ESP32 code should be part of the appendix, the code listings are included here for reference.

B.1 `esp32.ino`

Path: `esp32/esp32.ino`

B.2 Headers

The following listings contain all header files from the `esp32/` project root and source tree. Note that `wifi_mqtt_secrets_template.txt` is included instead of the actual secrets file for security reasons.

B.3 Source Files

The following listings contain all source files from the `esp32/` project root and source tree.